

Self-Consistency

Módulo 6 · Ingeniería de Prompts

Múltiples cadenas de razonamiento, votación por mayoría, estimación de confianza y escalada inteligente a revisión humana.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Self-Consistency

En 2022, Wang et al. publicaron "Self-Consistency Improves Chain of Thought Reasoning in Language Models", introduciendo una técnica que usa la consistencia entre múltiples respuestas como señal de confianza. La idea es elegante: si el modelo genera el mismo resultado a través de múltiples cadenas de razonamiento independientes, hay mayor confianza en que ese resultado es correcto. Es el equivalente estadístico de la sabiduría colectiva: si 8 de 10 personas independientes que razonan sobre el mismo problema llegan a la misma conclusión, esa conclusión es más confiable.

Self-consistency no requiere acceso a las probabilidades internas del modelo ni a información adicional. Es completamente unsupervised, funciona con cualquier LLM, y no requiere fine-tuning ni modificaciones al modelo. Solo requiere llamar al modelo múltiples veces con temperatura >0 (para obtener diversidad) y agregar los resultados por mayoría.

El mecanismo de self-consistency

El proceso de self-consistency tiene tres pasos. Primero, generar múltiples cadenas de razonamiento diversas para el mismo problema usando CoT con temperatura moderada-alta (0.7-1.0) para obtener variedad. El número de muestras típico es entre 5 y 40, con 10-20 siendo el rango más común en la práctica. Segundo, extraer la respuesta final de cada cadena de razonamiento. Tercero, seleccionar la respuesta más frecuente (mayoría) como la respuesta final del sistema.

La intuición probabilística: si el modelo tiene 60% de probabilidad de razonar correctamente en una sola muestra, con 10 muestras independientes la probabilidad de que la mayoría sea correcta sube dramáticamente (ley de los grandes números aplicada al espacio de razonamientos). Wang et al. reportaron mejoras de 10-20 puntos porcentuales en benchmarks de razonamiento aritmético y de sentido común usando self-consistency sobre CoT estándar.

Self-consistency para estimación de confianza

Más allá de mejorar la precisión, self-consistency proporciona una estimación de confianza: la fracción de muestras que coinciden con la respuesta más frecuente. Si 9 de 10 muestras dan la misma respuesta, la confianza es alta. Si 6 de 10 dan una respuesta y 4 otra, la confianza es baja y el caso merece revisión humana. Esta propiedad es especialmente valiosa para sistemas de toma de decisiones en producción: permite descartar automáticamente los casos de alta incertidumbre al circuito de revisión humana.

Implementación y trade-offs

La implementación básica de self-consistency requiere: un prompt con CoT (few-shot o zero-shot), N llamadas al modelo con temperatura 0.7-1.0 para obtener N respuestas independientes, extracción de la respuesta final de cada llamada (puede requerir parsing del

output), y conteo de frecuencias para identificar la respuesta mayoritaria.

El trade-off es claro: self-consistency con $N=10$ es 10x más caro que una sola llamada. El beneficio (mejora de precisión del 10-20%) debe justificar ese coste. Las reglas prácticas: usa self-consistency cuando el coste de un error es alto (decisiones de alto impacto), cuando el modelo tiene una tasa de error significativa en la tarea con una sola muestra, y cuando el volumen de consultas lo hace económicamente viable.

Universal Self-Consistency

Una extensión de self-consistency para tareas de generación libre (donde las respuestas no son discretas y no se puede hacer un conteo de mayoría simple) es Universal Self-Consistency: se generan N respuestas y luego se usa un LLM para seleccionar o sintetizar la respuesta más consistente entre todas ellas. El LLM evaluador recibe todas las respuestas y elige la más representativa o genera una síntesis que captura los puntos de mayor consenso.

SECCIÓN 4 · EJEMPLOS REALES

- Clasificación de riesgo legal con self-consistency: un sistema que clasifica documentos legales en categorías de riesgo (bajo, medio, alto, crítico) usa self-consistency con $N=10$. Solo actúa automáticamente si la mayoría es $>8/10$. Para los casos donde la mayoría es 6-8/10, escala a revisión de un abogado. Esto produce un sistema con alta precisión en los casos de actuación automática y gestión correcta de la incertidumbre.
- Diagnóstico médico de apoyo: un sistema de apoyo a la decisión clínica usa self-consistency con $N=15$ sobre los síntomas y signos del paciente. La confianza (fracción de acuerdo) se reporta junto con el diagnóstico sugerido. El médico recibe tanto la sugerencia como una indicación de cuánta certeza tiene el sistema, permitiendo calibrar el peso que da a la sugerencia.
- Estimación de esfuerzo en gestión de proyectos: para estimar el esfuerzo de un story de usuario, el sistema genera $N=7$ estimaciones independientes con CoT. Si la varianza entre estimaciones es baja (convergencia), la estimación es confiable. Si la varianza es alta, el story necesita más refinamiento antes de poder estimarse.
- Detección de alucinaciones con self-consistency: se usa self-consistency no para encontrar la respuesta correcta sino para detectar cuándo el modelo alucina: si el modelo da respuestas distintas en cada muestra para la misma pregunta factual, es señal de incertidumbre factual (posible alucinación). Alta varianza entre muestras = baja confianza = alto riesgo de alucinación.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar temperatura 0 para self-consistency. Con temperatura 0, el modelo produce el mismo output determinista en cada muestra. Se necesita temperatura > 0 (típicamente 0.7-1.0) para obtener diversidad entre las muestras y que el mecanismo de mayoría tenga sentido.

Error 2: Aplicar self-consistency a preguntas con respuestas que requieren consenso. Self-consistency asume que la respuesta correcta es la misma para todos los caminos de razonamiento. Para preguntas de opinión, preferencia o interpretación donde pueden

existir múltiples respuestas igualmente válidas, la mayoría puede no reflejar la "mejor" respuesta.

Error 3: Ignorar el coste multiplicativo. N=20 significa 20 llamadas al API con el coste y latencia correspondientes. Para sistemas de alta frecuencia, self-consistency puede ser prohibitivamente caro. Evalúa si la mejora de precisión justifica el coste para cada caso de uso específico.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Guía de decisión para cuándo usar self-consistency: el coste del error es alto (decisiones irreversibles o con consecuencias importantes) → self-consistency con N=10-20. La tasa de error del modelo en la tarea con N=1 es >10% → self-consistency mejora significativamente. El volumen es bajo o medio (no millones de consultas al día) → el coste adicional es manejable. Para alto volumen con bajo coste de error, una sola llamada con CoT es suficiente.

La implementación más limpia en Python: genera N respuestas en paralelo (asyncio o threading para minimizar latencia), extrae la respuesta de cada una, cuenta la frecuencia de cada respuesta única, devuelve (respuesta_mayoritaria, fraccion_acuerdo). Con fraccion_acuerdo como métrica de confianza, el sistema puede decidir automáticamente si actuar o escalar.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T2 (CoT): self-consistency se aplica sobre cadenas de razonamiento CoT para mejorar la precisión.
- M5-T7 (Evaluación de LLMs): self-consistency es una herramienta de evaluación de confianza además de una técnica de mejora de precisión.
- M8 (LLMOps): la escalada a revisión humana basada en la fracción de acuerdo de self-consistency es un patrón de LLMOps para gestionar la incertidumbre en producción.

SECCIÓN 8 · RESUMEN EJECUTIVO

Self-consistency genera N respuestas independientes con CoT y selecciona la más frecuente (mayoría) como respuesta final. Mejora la precisión en razonamiento en 10-20 puntos porcentuales sobre CoT estándar, sin fine-tuning ni acceso a probabilidades internas. La fracción de acuerdo entre muestras es una estimación de confianza: alta fracción = alta confianza, baja fracción = incertidumbre → escalar a revisión humana. El coste es N veces el de una sola llamada: justificado cuando el coste del error es alto. Implementación: N llamadas en paralelo con temperatura 0.7-1.0, extracción de respuesta, conteo de mayoría.